Manual técnico

1. Tecnologías y librerías

Elemento	Función dentro de Mercadito
Python 3	Lenguaje de programación utilizado en la aplicación.
Django 5.2.18	Framework web: modelos, formularios, vistas, autenticación, rutas, plantillas y administración.
PostgreSQL	Base de datos normal de desarrollo: almacena mercados, zonas, usuarios, carritos, pedidos y productos.
psycopg 3 / psycopg-binary	Conector empleado por Django para comunicarse con PostgreSQL.
python-dotenv	Carga las variables de conexión de PostgreSQL desde el archivo .env.
HTML y CSS	Presentación web, navegación, controles de cantidad, formularios y diseño adaptable.
Git y GitHub	Versionado, repositorio colaborativo y publicación de avances.

Versiones: requirements.txt especifica Django==5.2.18 y psycopg 3.3.6 (psycopg y psycopg-binary), entre otras dependencias.

2. Organización y flujo de la aplicación

El proyecto utiliza el patrón de Django: el navegador envía solicitudes a las rutas de tienda/urls.py; views.py recibe y valida solicitudes; forms.py valida la entrada del usuario; servicios.py coordina las operaciones de compra; models.py representa las entidades guardadas en PostgreSQL; y las plantillas de templates/tienda/ muestran el resultado. La carpeta static contiene la hoja de estilos.

La autenticación se basa en el sistema de usuarios de Django; la sesión conserva la zona elegida. La aplicación mantiene un carrito distinto para cada combinación usuario–mercado.

3. Archivos principales del proyecto

Archivo / directorio	Responsabilidad
manage.py	Comandos de Django: migraciones, pruebas, servidor local y administración.
mercadito/settings.py	Aplicaciones, autenticación, zona horaria, middleware y conexión .env/PostgreSQL.
mercadito/urls.py	Incluye las rutas principales de la aplicación tienda.
tienda/models.py	Entidades, métodos de cálculo, reglas y relaciones de datos.
tienda/forms.py	RegistroForm, IngresoForm, CantidadForm y ConfirmarPedidoForm.
tienda/validaciones.py	Funciones dinero() y validar_cantidad().
tienda/servicios.py	Acciones del carrito y confirmación transaccional de la compra.
tienda/views.py / urls.py	Navegación, permisos, controladores web y rutas.
tienda/middleware.py	Respuesta 503 comprensible ante determinados errores de base de datos.
tienda/templates/tienda/	Pantallas de compra, registro, carrito, pedidos y detalle.
tienda/static/tienda/css/	Estilos visuales del sitio Mercadito.
tienda/migrations/0002_completar_compra.py	Segunda migración: identificador de confirmación y restricciones.
tienda/tests.py	Pruebas de modelos, formularios, rutas, permisos y operaciones.
tienda/fixtures/datos_iniciales.json	Mercados, zonas, categorías y datos de prueba iniciales.

4. Instalación inicial en una computadora nueva

Requisitos: Git, Python 3 y PostgreSQL con pgAdmin. Aceptar la invitación de colaborador si corresponde. Abrir PowerShell en una ubicación de trabajo y ejecutar:

```
git clone https://github.com/Mafrezz/Mercadito.git
cd Mercadito
python -m venv venv
.\venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
```

En pgAdmin → Query Tool, crear el usuario y la base de datos (adaptando la contraseña):

```
CREATE USER mercadito_user WITH PASSWORD 'tu_contrasena';
CREATE DATABASE mercadito OWNER mercadito_user
  ENCODING 'UTF8' TEMPLATE template0;
ALTER USER mercadito_user CREATEDB;
```

Después, copiar el archivo de ejemplo de configuración y completar DB_PASSWORD con la contraseña del usuario mercadito_user:

```
Copy-Item .env.example .env
$env:PYTHONUTF8 = "1"
python manage.py migrate
python manage.py loaddata datos_iniciales
python manage.py createsuperuser
python manage.py runserver
```

El fixture de datos iniciales solo se ejecuta al preparar una base nueva. En una base ya utilizada NO se vuelve a ejecutar loaddata porque podría sobrescribir información editada.

Abrir http://127.0.0.1:8000/ para utilizar la página, o http://127.0.0.1:8000/admin/ para acceder al panel de administración.

5. Actualizar una copia ya instalada

Si Mercadito ya está instalado y funcionando, conservar .env, venv y .git. Detener runserver con Ctrl+C, respaldar el proyecto y la base de datos, sincronizar el código y aplicar las migraciones nuevas:

```
git status
git pull origin main
.\venv\Scripts\Activate.ps1
python manage.py check
python manage.py migrate
python manage.py runserver
```

Antes de git pull deben guardarse, integrarse o respaldarse los cambios locales. Los comandos se ejecutan dentro de la carpeta que contiene manage.py. Si el código trae migraciones nuevas, python manage.py migrate las aplica (por ejemplo, tienda/migrations/0002_completar_compra.py).

6. Configuración de PostgreSQL y seguridad de archivos

La conexión se define en mercadito/settings.py con DB_NAME, DB_USER, DB_PASSWORD, DB_HOST y DB_PORT. El archivo .env contiene la contraseña de la base de datos, está excluido mediante .gitignore y no debe subirse al repositorio; .env.example sí sirve como ejemplo de configuración.

En esta fase la aplicación se ejecuta de forma local con DEBUG=True, por lo que su configuración es de desarrollo académico y no constituye por sí sola una publicación segura en Internet.

7. Clases y relaciones de la base de datos

Cada clase de tienda/models.py es un modelo de Django que representa una tabla. A continuación, se resumen las once entidades propias de la aplicación, además del modelo de usuario integrado en Django.

Modelo	Atributos y finalidad principal
Zona	numero y municipio: identifica zonas de Guatemala y Mixco.
Mercado	nombre, direccion, zona, horario, activo y zonas_entrega: registra mercados y su cobertura.
MercadoZona	mercado, zona y tarifa_envio: relación mercado–zona con costo fijo de entrega.
Categoria	nombre y orden: clasificación del catálogo.
Producto	nombre, categoria, unidad y descripcion: producto y su presentación.
ProductoMercado	producto, mercado, precio y disponible: oferta y precio específicos de cada mercado.
Direccion	usuario, alias, zona, direccion y referencias: entidad de direcciones asociadas a usuarios.
Carrito	usuario, mercado y creado: un carrito por usuario y mercado.
ItemCarrito	carrito, producto y cantidad: productos seleccionados antes de la compra.
Pedido	usuario, mercado, zona, direccion_entrega, fecha_entrega, subtotal, costo_envio, total, metodo_pago, estado, creado y token_confirmacion.
ItemPedido	pedido, producto, cantidad y precio_unitario: copia el precio histórico al confirmar.
Usuario de Django	username, nombre, apellidos, correo y contraseña cifrada mediante hash; provisto por Django.

Relaciones: un Mercado entrega en varias Zona por medio de MercadoZona; un Producto posee precios distintos en cada Mercado mediante ProductoMercado; un usuario tiene Carrito y Pedido; cada carrito tiene ItemCarrito y cada pedido contiene ItemPedido. Las restricciones evitan duplicar una combinación de usuario–mercado o un producto dentro de un mismo carrito.

8. Métodos y propiedades implementados

Clase	Métodos / propiedades	Aplicación práctica
Mercado	obtener_cobertura(zona)	Comprueba mercado activo y cobertura de la zona.
Producto	permite_fracciones, paso_cantidad, maximo_cantidad	Permite decimales en libras/litros y enteros en unidades/docenas.
ProductoMercado	calcular_importe(cantidad)	Multiplica cantidad validada por el precio del mercado.
Carrito	esta_vacio(), detalle_items(), calcular_subtotal(), calcular_envio(zona), calcular_total(zona)	Consulta renglones y obtiene subtotales y montos para el cliente.
ItemCarrito	clean(), precio_unitario, subtotal	Verifica cantidad y usa el precio actual de su mercado.
Pedido	clean(), calcular_subtotal(), calcular_total(), actualizar_totales()	Valida dirección/fecha inicial y recalcula importes desde ItemPedido.
ItemPedido	subtotal	Cantidad × precio unitario guardado al comprar.

El método __str__() también se define en los modelos para identificar sus objetos de forma legible en Django y en el administrador.

- ## Instanciación de objetos

Los objetos se crean y se manejan durante la ejecución del programa, no solo al cargar los datos: cada acción del usuario instancia las clases del proyecto.

- Carrito.objects.get_or_create(usuario=..., mercado_id=...) obtiene o crea el carrito del usuario en el mercado elegido (función agregar_producto de servicios.py).
- ItemCarrito(carrito=..., producto=..., cantidad=...) se instancia cuando el producto aún no está en el carrito; si ya existe, se modifica la cantidad del renglón.

- Pedido(usuario=..., mercado=..., zona=..., direccion_entrega=..., fecha_entrega=..., subtotal=..., costo_envio=..., total=..., token_confirmacion=...) se instancia en confirmar_compra(), se valida con full_clean() y se guarda con save().
- ItemPedido(pedido=..., producto=..., cantidad=..., precio_unitario=...) se instancia una vez por cada renglón del carrito, copiando el precio vigente al momento de la compra.
- RegistroForm, CantidadForm y ConfirmarPedidoForm se instancian en views.py con los datos recibidos (por ejemplo, RegistroForm(request.POST)) para validarlos antes de guardarlos.
- Zona, Mercado, MercadoZona, Categoria, Producto y ProductoMercado se instancian al cargar datos_iniciales.json con loaddata y desde el panel de administración.

9. Cálculo de precios, envío y total

Se utiliza Decimal, no float, y la función dinero() redondea a dos cifras decimales. El precio de cada artículo se consulta en ProductoMercado según el mercado elegido.

Subtotal de artículo = cantidad × precio_unitario

Subtotal del carrito = suma de subtotales de artículos

Total = subtotal del carrito + tarifa de envío de la zona

Ejemplo con los datos de demostración: 3 libras de manzana a Q8.00 por libra en el Mercado El Guarda producen Q24.00 de subtotal. La tarifa de envío a la zona 3 de Guatemala es Q15.00, por lo que el total es Q39.00. Los valores reales dependen de la oferta y la cobertura elegidas.

Cuando se confirma el pedido, el sistema guarda en ItemPedido el precio vigente en ese momento. Las compras históricas conservan sus valores aunque después se modifique el catálogo.

10. Vistas, rutas y pantallas

Todas las rutas indicadas son relativas a http://127.0.0.1:8000/. Las acciones que cambian datos se tramitan con POST.

Vista	Ruta	Propósito
inicio	/	Pantalla de bienvenida y selector de zonas.
mercados	/mercados/	Mercados activos de la zona y tarifa de envío.
catalogo	/mercado/<id>/	Catálogo filtrable por categorías.
registro	/registro/	Crear usuario con datos y contraseña confirmada.
IngresarView	/ingresar/	Inicio de sesión con autenticación de Django.
cerrar_sesion	/salir/	Salida de sesión por solicitud POST.
carritos	/carritos/	Lista de carritos con productos del usuario.
ver_carrito	/carrito/mercado/<id>/	Detalle del carrito de un mercado.
agregar	/carrito/agregar/<id>/	Agregar oferta y cantidad al carrito.
cambiar_cantidad	/carrito/item/<id>/cantidad/	Actualizar cantidad de un renglón.
quitar	/carrito/item/<id>/quitar/	Quitar un producto del carrito.
vaciar	/carrito/<id>/vaciar/	Vaciar todos los productos de un carrito.
confirmar	/carrito/<id>/confirmar/	Dirección, fecha y confirmación de compra.
pedidos	/pedidos/	Historial personal (10 pedidos por página).
detalle_pedido	/pedido/<id>/	Detalle y estado de un pedido propio.

Los identificadores <id> representan el identificador numérico del objeto correspondiente. Las rutas exactas aparecen en tienda/urls.py.

- Widgets de la interfaz

La interfaz es web: los widgets son los elementos HTML de las plantillas de tienda/templates/tienda/, algunos generados por los formularios de Django (forms.py).

- Botones: «Ver mercados», «Ver productos», «Agregar», «Actualizar», «Quitar», «Vaciar este carrito», «Continuar pedido», «Confirmar pedido» y «Cerrar sesión».
- Etiquetas (label) asociadas a cada campo, por ejemplo «Tu zona», «Nombres», «Apellidos» y «Correo electrónico».
- Entradas de datos: campos de texto (usuario, nombres y apellidos), correo electrónico, contraseñas, campo numérico de cantidad (con mínimo, paso y máximo según la unidad del producto), área de texto para la dirección de entrega y selector de fecha con fecha mínima de hoy.
- Lista desplegable: el selector «Tu zona» de la pantalla de bienvenida, con las zonas agrupadas por municipio.
- Marcos y contenedores: formularios, fichas de mercado, secciones por categoría, barras de navegación y mensajes de aviso o confirmación.
- Campo oculto: la confirmación firmada del pedido.

Navegación: los enlaces «Cambiar zona», «Cambiar mercado» y «Mis pedidos», junto con la barra de navegación de base.html, permiten pasar entre las secciones según las rutas de la tabla anterior.

11. Registro, carrito y confirmación

• Registro e inicio de sesión

RegistroForm extiende UserCreationForm e incorpora nombres, apellidos y correo; verifica las reglas de contraseña proporcionadas por Django. Una vez creado el usuario, se muestra un mensaje de éxito y se lo dirige a iniciar sesión. IngresoForm usa AuthenticationForm. Cerrar sesión no borra los carritos ni los pedidos del cliente.

• Carrito: agregar, modificar, quitar y vaciar

agregar_producto() crea el carrito si no existe y suma la cantidad cuando el producto ya está incluido. actualizar_item(), quitar_item() y vaciar_carrito() modifican los registros en PostgreSQL. Todas estas acciones verifican la propiedad del carrito y usan operaciones atómicas; al consultar se muestran los precios actuales y se advierte si una oferta dejó de estar disponible.

• Confirmación del pedido

ConfirmarPedidoForm solicita dirección de entrega y fecha. confirmar_compra() vuelve a comprobar usuario, zona, mercado, artículos, disponibilidad, cantidades, precios y tarifa. Usa un token firmado con huella del resumen y vigencia de dos horas, más un identificador único para reconocer reenvíos y evitar pedidos duplicados.

Dentro de transaction.atomic() se crea Pedido, se agregan sus ItemPedido, se recalculan los importes y se vacía el carrito. Si falla el guardado, la transacción revierte sus escrituras: no queda una compra parcial. No se utiliza una pasarela de pago; el pedido se registra como pago contra entrega.

• Historial y detalle

La vista pedidos() filtra las compras por el usuario autenticado y presenta el historial paginado. detalle_pedido() permite consultar el mercado, la dirección, la fecha, los productos, el costo de envío, el total y el estado del pedido. No permite consultar pedidos de otros usuarios.

12. Validaciones, avisos y manejo de excepciones

Situación	Respuesta del sistema
Zona no indicada o inexistente	Se informa el error y se vuelve al selector; no se muestra un mercado no cubierto.
Usuario o contraseña inválidos	AuthenticationForm informa que los datos no son correctos.
Registro incompleto o usuario repetido	El formulario señala el campo que debe corregirse.
Cantidad nula, negativa, demasiado grande o con formato inválido	CantidadForm rechaza entradas fuera de 0.01 a 9999.99 o con más de dos decimales.
Fracciones en productos por unidad o docena	validar_cantidad() exige cantidades enteras.

Situación	Respuesta del sistema
Producto retirado o mercado inactivo	La compra se detiene y solicita revisar la oferta o la cobertura.
Dirección demasiado breve o fecha anterior a hoy	Se muestran mensajes y se conservan los datos útiles del formulario.
Resumen modificado o confirmación alterada/vencida	Se solicita revisar los nuevos precios o la tarifa antes de confirmar.
Error de base de datos	Se muestra mensaje controlado; el middleware puede emitir una página 503.
Acceso a carrito/pedido ajeno	Se deniega al filtrar por usuario; los cambios requieren inicio de sesión.

Los formularios que modifican información usan protección CSRF. Las vistas aceptan POST en las acciones de cambio y aplican login_required a carrito y pedidos. Los mensajes de éxito/error ayudan al usuario; tras guardar correctamente se redirige a otra pantalla, evitando el envío repetido al refrescar.

13. Persistencia en PostgreSQL y migraciones

Lectura: la aplicación consulta las zonas, mercados, coberturas, productos, precios y pedidos desde PostgreSQL. Escritura: se guardan usuarios, artículos del carrito y pedidos directamente desde formularios y vistas; los administradores pueden modificar las ofertas y los estados de los pedidos.

La migración inicial 0001_initial.py crea las tablas del proyecto. La migración 0002_completar_compra.py agrega Pedido.token_confirmacion (UUID único, nulo para los pedidos antiguos), actualiza validadores decimales y añade restricciones de cantidades positivas, importes no negativos y totales consistentes. Las migraciones se aplican con python manage.py migrate y no eliminan los registros existentes.

Para ver los datos en pgAdmin, abrir Databases → mercadito → Schemas → public → Tables y usar View/Edit Data → All Rows. También se puede abrir Query Tool y consultar, por ejemplo:

```
SELECT * FROM auth_user LIMIT 20;
SELECT * FROM tienda_carrito LIMIT 20;
SELECT * FROM tienda_itemcarrito LIMIT 20;
SELECT * FROM tienda_pedido ORDER BY creado DESC LIMIT 20;
SELECT * FROM tienda_itempedido LIMIT 20;
```

Para revisar la integridad, es preferible utilizar SELECT y evitar editar manualmente los totales del pedido: esos cálculos deben realizarse en Python, mediante los métodos del proyecto.

Maria Fernanda Alvarez Morales 202001537

Sergio Luis Valdéz Pérez 202111468

Curso: Proyectos de Computación aplicados a I.E.

Proyecto: Mercadito